

CNuvS Ex. 3 - Jan Lindauer, Paul Feidieker

Task 4

(a)

Key characteristic	Given Description	IPv4	IPv6
<i>Notation</i>	Hexadecimal (e.g. 2001:db8::1)	☐	☒
<i>Address length</i>	32-bit	☒	☐
<i>Helper protocol</i>	ARP	☒	☐
<i>Header structure</i>	Additional Fields in the header: Traffic Class, Flow Label, Payload Length, Next Header, Hop Limit. Header size is fixed to 40 bytes.	☐	☒

[1, p.]

(b) Firstly NDP uses IPv6, contrary to ARP using IPv4. Secondly the MAC address resolution requests are sent as broadcasts for ARP protocol and NDP only sends them to its nearest neighbors which means it is considered a multicast message. [2]

(c) 1234:0000:0000:00ab:1234 -> 1234::ab:1234 [1, p. 32]

(d) 0010:4aee:53ab -> 0210:4aff:feee:53ab [1, p. 32]

(e) The solution is called tunneling. It is a technique to encapsulate IPv6 packets within IPv4 packets, allowing them to be transmitted over an IPv4 network. [1, p. 33]

Task 5

(a)

Link Interface	Destination Address Range
0	10101110 01011110 11000000 00000000 through 10101110 01011110 11000111 11111111
1	10101110 01011110 11001100 10000000 through 10101110 01011110 11001100 11111111
2	10101110 01011110 11001100 00000000 through 10101110 01011110 11001101 11111111
3	otherwise

Link Interface	Prefix Match
0	10101110 01011110 11000
1	10101110 01011110 11001100 1
2	10101110 01011110 1100110
3	otherwise

- (b) 174.94.205.60 -> Link Interface 2
 Since the address has the longest (only) prefix match with the address range of link interface 2 it belongs to this link interface. Note that link interface 1 defines a subnet of link interface 2 but the prefix doesn't match the address.
- (c) 174.94.194.175 -> Link Interface 0
 This address only matches the prefix of link interface 0, therefore it belongs to that interface.
- (d) 174.94.206.127 -> Link Interface 3
 This address doesn't match any of the well defined link interfaces. It conflicts link interface at the 21st prefix digit. For link interface 1 and 2 it is prefix digit 23. Therefore only link interface 3 remains.

Task 6

- (a) **Subnet mask:** 255.255.252.0 (decimal), 11111111 11111111 11111100 00000000 (binary) **Network part:** 172.16.4.0 (decimal), 10101100 00010000 00000100 00000000 (binary) **Host part:** 0.0.1.10 (decimal), 00000000 00000000 000000 01 00001010 (binary)
 Only the marked parts of the binary representation really make out the network and host part of the subnet respectively.

[1, p. 4, 5]

- (b) So the first 23 bit of the address is the network part of the address. Therefore the last 9 bit of the address are the host part and can be used as addresses inside of the network, although we have to correctly subtract the broadcast and network address. Our total of usable addresses sums up to:

$$510 = 512 - 2 = 2^9 - 2$$

[1, p. 5]

- (c) The 127.0.0.0/8 block is called the loopback block. It is used to address the same host the message is coming from. It is often called localhost.

[3]

Task 7

- (a) The main problem was a shortage of IP addresses. The end-to-end-principle suggested that every device should have a unique IP address, but with the growth of the internet and the available IP

addresses in the IPv4 protocol (2^{32}) this became a problem. NAT was a solution to this by giving local networks only one IP address to the outside “world”. However this solution was contrary to the end-to-end-principle and caused problems for applications that needed to establish direct connections between devices. The solution to solve this issue was the implementation of the IPv6 protocol which has a much larger address space (2^{128}).

[1, p. 17, 18, 21]

(b) Request:

10.0.0.4:3345 → 1.1.1.1:80

Response:

1.1.1.1:80 → 138.76.29.7:5001

[1, p. 20]

Task 8

(a) The four addresses can be aggregated to 172.16.1.0/24.

The aggregation is possible because they only differ on bits 25 and 26 (last two bits of the /26 network block). Since the 4 addresses cover all possible combinations of bits 25 & 26 (00, 01, 10, 11) and share the same interface, they can be aggregated to the /24 block.

[1, p. 4, 18]

(b) For interface #1 the following calculations can be done to determine the routing table entry:

First Range:

10.40.50.59 → 00001010 00101000 00110010 00111011

10.60.103.225 → 00001010 00111100 01100111 11100001

As the marked bits are the same for both addresses, we can aggregate them to the address range 10.32.0.0/11.

Second Range:

10.0.20.8 → 00001010 00000000 00010100 00001000

10.23.150.252 → 00001010 00010111 10010110 11111100

As the marked bits are the same for both addresses, we can aggregate them to the address range 10.0.0.0/11.

To see, if the range can be further aggregated, we can compare the two calculated address blocks:

10.32.0.0 → 00001010 001 00000 00000000 00000000

10.0.0.0 → 00001010 000 00000 00000000 00000000

As the marked bits are the same for both addresses, we can aggregate them to the address range 10.0.0.0/10.

So the final routing table entry for interface #1 is: 10.0.0.0/10

For interface #2 the following calculations can be done to determine the routing table entry:

192.168.10.128 → 11000000 10101000 00001010 10000000

192.168.10.255 → 11000000 10101000 00001010 11111111

As the marked bits are the same for both addresses, we can aggregate them to the address 192.168.10.128/25.

So the final routing table entry for interface #2 is: 192.168.10.128/25

[1, p. 13, 14]

Sources

- [1] Prof. Dr. Marco Zimmerling, “Computer Networks & Distributed Systems Chapter 3: Internetworking.” 2026.
- [2] Nawaz Dhandala, [Online]. Available: <https://oneuptime.com/blog/post/2026-03-20-ndp-address-resolution-vs-arp/view>
- [3] Kolade Chris, [Online]. Available: <https://www.freecodecamp.org/news/what-is-localhost/>